

数据科学导论

EdgeThemis: 基于形式化验证的 因果推理引擎

A Causal Reasoning Engine with Formal Verification

姓名: 崔鹏宇

学号: 2309404018

专业: 人工智能

日期: 2026 年 6 月

摘要

大语言模型 (LLM) 在因果推理任务中存在严重的“拓扑幻觉”问题：生成的因果图可能包含逻辑环路、忽略混淆因子、或产生虚假的因果关联。本文介绍 EdgeThemis——一个专为边缘硬件设计的混合架构因果推理引擎。该系统将形式化数学验证 (Rust) 无缝嵌入大模型推理循环 (Python 状态机)，通过三大核心机制保障因果图的逻辑一致性：(1) Kahn 拓扑排序算法检测环路；(2) Bayesian Ball 算法验证 d-分离条件独立性；(3) FMEA (失效模式与影响分析) 工业风险评分量化每条因果边的风险等级。系统还引入了自反思纠错闭环，当验证失败时将拦截报告反馈给 LLM 进行自动修正，最多 5 轮熔断。消融实验表明，完整系统 (Config C) 在 DAG 合法性、d-分离一致性、FMEA 标注和自纠错能力四个维度均显著优于纯 LLM 基线。本系统基于 MSR (2023) 关于 LLM 因果推理缺陷的研究和 Reflexion (NeurIPS 2023) 的自反思框架，首次将工业 FMEA 方法论与 LLM 因果推理结合，在 8GB VRAM 的 RTX 4060 边缘硬件上实现了工业级的因果推理保障。

关键词：因果推理；形式化验证；大语言模型；d-分离；FMEA；边缘计算

目录

1	引言	3
1.1	研究背景	3
1.2	相关工作	3
1.3	本文贡献	3
2	系统架构	4
2.1	整体流水线	4
2.2	Generator 节点	4
2.3	Validator 节点	5
2.4	Reflector 节点	5
3	核心算法	5
3.1	Kahn 环路检测	5
3.2	Bayesian Ball d-分离算法	5
3.2.1	方向状态	6
3.2.2	四条状态转移规则	6
3.2.3	六种判定情况	6
3.3	FMEA 风险评分	7
4	自反思纠错闭环	7
4.1	闭环流程	7
4.2	熔断机制	7
4.3	定向修复	8
5	实验与分析	8
5.1	实验配置	8
5.2	消融实验	8
5.3	案例分析	9
6	结论	9
A	附录 A: Rust 核心代码	11
A.1	dag.rs —紧凑邻接表	11
A.2	algorithms.rs —Kahn 环路检测（关键片段）	11
A.3	algorithms.rs —Bayesian Ball d-分离（关键片段）	12
A.4	fmea_evaluator.rs —FMEA RPN 评分	14

B 附录 B: Python 管线代码	16
B.1 agent_state_machine.py —数据模型	16
B.2 nodes.py —Validator 节点（关键片段）	16
B.3 app.py —LangGraph 状态机组装（关键片段）	17

1 引言

1.1 研究背景

因果推理是科学发现和工程决策的基石。在医疗诊断、工业故障分析、自动驾驶等领域，准确识别变量之间的因果关系直接关系到决策的正确性和安全性。近年来，大语言模型（Large Language Models, LLM）展现出了从文本中提取因果关系的潜力，但其输出的可靠性面临严峻挑战。

微软研究院（Microsoft Research）在 2024 年 ICLR 发表的工作 [1] 中，通过构建 Corr2Cause 数据集，残酷地证明了一个事实：**LLM 本质上是“因果复读机”（Causal Parrots）**——它们只能记住训练语料中的词汇共现模式，一旦面对抽象变量构成的因果问题，表现便退化至随机猜测水平。微调（Fine-tuning）虽然能提升特定数据集上的分数，但面对分布外（OOD）数据时立刻原形毕露。

1.2 相关工作

在因果推理增强领域，现有方法可分为以下几个流派：

- (1) **语言反思派**：Reflexion[2]（NeurIPS 2023）提出了 Actor→Evaluator→Reflection 的语言强化学习闭环，用自然语言给出强化信号让 LLM 自我纠错。然而，其 Evaluator 是“软性的”——用语言判断对错，无法保证数学绝对确定性。
- (2) **多模型辩论派**：CRAwDAD[4] 让两个推理大模型互相辩论直至共识。但这种方案需要并发运行多个大参数模型，在 8GB 显存的边缘硬件上毫无落地可能。
- (3) **工具调用派**：Causal Agent[5] 让 LLM 调用外部 Python 因果库处理表格数据。但若 LLM 在规划链条上产生逻辑幻觉，外部工具再精确也会被带入歧途。

1.3 本文贡献

EdgeThemis 走的是第四条路线——**神经符号学（Neuro-Symbolic）**：用 LLM 做它最擅长的事（从文本中提取因果节点和边），然后直接启动 Rust 底层物理状态机，用编译期的 Kahn 算法和 Bayesian Ball 算法执行 100% 确定性的数学审判。本文的主要贡献包括：

- (1) **混合架构**：LLM 结构化提取（JSON Schema 强制输出）+ Rust 形式化验证引擎（Kahn 环路检测 + Bayesian Ball d-分离），实现跨语言的确定性因果验证。
- (2) **FMEA 风险量化**：将工业工程中的失效模式与影响分析（FMEA）融入因果图每条边，采用 $RPN = 100 \times S + 10 \times O + D$ 公式量化风险等级。

- (3) **自反思纠错闭环**: 验证失败时, 拦截报告反馈给 LLM 生成器进行定向修复, 最多 5 轮熔断, 形成“提取-验证-纠错”的自动闭环。

2 系统架构

2.1 整体流水线

EdgeThemis 采用三层异构架构, 如图 1所示。系统流水线为:

Input Text → **Generator (Python)** → **Validator (Rust)** → **Reflector (Python)**
→ **Output**

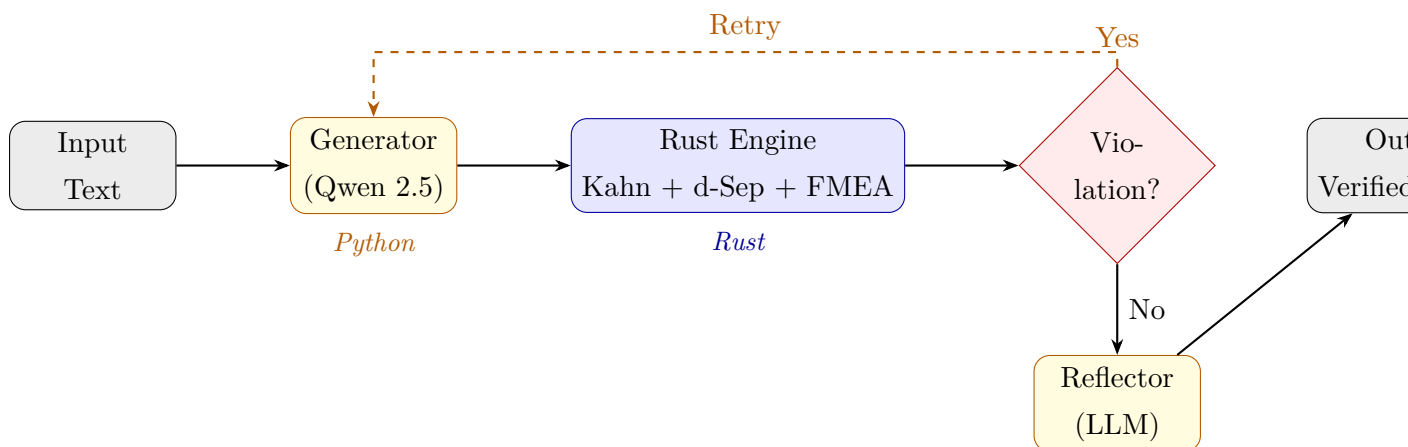


图 1: EdgeThemis 系统架构流水线

2.2 Generator 节点

Generator 接收自然语言文本, 通过 JSON Schema 约束 LLM 强制输出结构化的 CausalEdge 数据结构。每条边包含五个字段:

- **cause**: 原因节点名称
- **effect**: 结果节点名称
- **S (Severity)**: 严重度, 1–10
- **O (Occurrence)**: 发生概率, 1–10
- **D (Detection)**: 检测难度, 1–10

输出格式由 Pydantic 模型 `ExtractedGraph` 校验, 同时包含 `reasoning_process` (Chain-of-Thought 推理链) 字段, 确保 LLM 先推理再输出结论。

2.3 Validator 节点

Validator 是系统的“物理测谎仪”，包含三个并行执行的子模块：

- (1) **Kahn 环路检测**: $O(V + E)$ 拓扑排序，检测因果图中是否存在有向环。
- (2) **Bayesian Ball d-分离**: $O(V^3)$ 方向状态 BFS，全量扫描提取条件独立性断言。
- (3) **FMEA RPN 计算**: 对每条因果边独立执行风险评分。

2.4 Reflector 节点

Reflector 以“严谨审查员”人格运行，逐一审判 Validator 提取的 d-分离断言。仅在发现结构性逻辑错误时判决 REJECT，触发纠错循环。

3 核心算法

3.1 Kahn 环路检测

Kahn 算法通过拓扑排序检测有向图中是否存在环路。其核心思想是“剥洋葱”式层层拆解：

1. 统计每个节点的入度（被指向的次数）。
2. 将所有入度为 0 的“自由节点”推入队列。
3. 依次出队节点，将其所有邻居的入度减 1；若邻居入度变为 0 则入队。
4. 若成功拆解的节点数等于总节点数，则无环；否则存在环路。

时间复杂度为 $O(V + E)$ ，其中 V 为节点数， E 为边数。此外，系统在边注入阶段即拦截自环边 $v_i \rightarrow v_i$ ，从源头杜绝无意义的因果回路。

3.2 Bayesian Ball d-分离算法

d-分离（directional separation）是因果推理的核心判据，用于判断在给定观测集 Z 的条件下，节点 X 和 Y 是否条件独立。EdgeThemis 采用 Bayesian Ball 算法——一种带方向状态的变种 BFS。

3.2.1 方向状态

传统 BFS 只记录 `visited[node]`, 而 Bayesian Ball 记录 `visited[(node, direction)]`, 其中 `direction` 有两个取值:

- **Up (逆箭头)**: 球从当前节点逆着箭头方向移动, 寻找父节点 (原因)。
- **Down (顺箭头)**: 球从当前节点顺着箭头方向移动, 寻找子节点 (结果)。

同一节点在不同方向上可以被多次访问, 因为不同方向对应不同的因果结构判定规则。

3.2.2 四条状态转移规则

算法的核心是四条状态转移规则, 如表 1 所示。

表 1: Bayesian Ball 四条状态转移规则

进入方向	Z 状态	允许 Up?	允许 Down?	对应结构	物理含义
Up	未观测	✓	✓	Fork $X \leftarrow Z \rightarrow Y$	后门大开
Up	已观测	×	×	Fork $X \leftarrow Z \rightarrow Y$	后门关闭
Down	未观测	×	✓	Chain $X \rightarrow Z \rightarrow Y$	合法通路
Down	已观测	✓	×	Collider $X \rightarrow Z \leftarrow Y$	致命陷阱

3.2.3 六种判定情况

三种基本因果结构在观测/未观测两种状态下共产生六种判定情况, 如表 2 所示。

表 2: d-分离六种判定情况

结构	图示	Z 含 M?	判定	直觉解释
Chain	$X \rightarrow M \rightarrow Y$	是	$X \perp Y$ 阻断	M 被观测, 信息传递截断
Chain	$X \rightarrow M \rightarrow Y$	否	$X \not\perp Y$ 连通	唯一合法因果通路
Fork	$X \leftarrow M \rightarrow Y$	是	$X \perp Y$ 阻断	共同原因被控制
Fork	$X \leftarrow M \rightarrow Y$	否	$X \not\perp Y$ 连通	后门大开
Collider	$X \rightarrow M \leftarrow Y$	是	$X \not\perp Y$ 激活	Explaining away 陷阱
Collider	$X \rightarrow M \leftarrow Y$	否	$X \perp Y$ 阻断	天然绝缘

其中, Collider (对撞) 结构是最反直觉的: 观测到中间节点 M 反而激活了 X 和 Y 之间的虚假依赖关系, 这称为 **explaining away** 效应。例如, “业务能力” 和 “人脉关系” 本无关联, 但一旦知道某人 “已成合伙人” (M 被观测), 若发现其业务能力不行, 就能推断其靠关系。

3.3 FMEA 风险评分

FMEA (Failure Mode and Effects Analysis) 是工业工程中广泛使用的风险评估方法。EdgeThemis 将其引入因果推理，每条因果边携带三个维度的评分 (S/O/D, 取值 1-10)，风险优先数 RPN 计算公式为：

$$RPN = 100 \times S + 10 \times O + D \quad (1)$$

RPN 的最大值为 $100 \times 10 + 10 \times 10 + 10 = 1110$ 。系统采用线性加权公式，通过拉大严重度 S 的权重 (100 倍)，确保高危失效节点在监控中无处遁形。当满足以下任一条件时，系统触发高风险告警：

- $RPN > 500$
- $S \geq 9$ (致命单点)

4 自反思纠错闭环

自反思闭环是 EdgeThemis 的核心创新之一，其设计借鉴了 Reflexion[2] 的 Actor-Evaluator-Reflection 框架，但用 Rust 确定性验证器取代了软性语言评估器。

4.1 闭环流程

当 Validator 检测到违规 (环路或 d-分离违反) 时，拦截报告被反馈给 Generator LLM 进行定向修复：

1. **Round 1:** LLM 提取因果图 $\rightarrow$ Rust Kahn 检测到环路 $\rightarrow$ 拦截报告反馈。
2. **Round 2:** LLM 修正环路 $\rightarrow$ Rust Bayesian Ball 发现 d-分离违反 $\rightarrow$ 拦截报告反馈。
3. **Round 3:** LLM 再次修正 $\rightarrow$ 全部验证通过 $\rightarrow$ 输出最终因果图。

4.2 熔断机制

系统设置 5 次拦截上限 (Circuit Breaker)。每次成功生成的图谱被记录为 `best_graph`；当验证完全通过时更新。即使触发熔断，系统仍返回历史最佳图谱而非空结果，保障系统的鲁棒性。

4.3 定向修复

当 Reflector 判决 REJECT 时，系统将具体的 d-分离断言（最多 10 条）和拒绝理由注入 Generator 的重试 Prompt，并携带上一轮图谱的边列表。Generator 做增量修复而非从头重写，大幅提高纠错效率。

5 实验与分析

5.1 实验配置

实验环境配置如表 3 所示。

表 3: 实验环境配置

项目	配置
LLM	Qwen 2.5-3B (Q4 量化)
推理框架	llama-server, OpenAI-compatible API
上下文窗口	4096 tokens
KV Cache	Q8_0 量化
GPU	RTX 4060 (8GB VRAM)
测试场景	医院感染链路、工业故障传播

5.2 消融实验

为验证各模块的增量贡献，设计三组消融对比实验，结果如表 4 所示。

表 4: 消融实验对比

评估指标	Config A: 纯 LLM	Config B: +Schema+Rust	Config C: +Reflection
DAG 合法性	× 可能含环路	✓ 环路被拦截	✓ 环路被拦截
d-分离一致性	× 未验证	△ 检测不修正	✓ 检测 + 自动修正
FMEA 风险标注	× 无	✓ 有	✓ 有
自纠错能力	× 无	× 无	✓ 最多 5 轮
综合图质量	低	中	高

消融实验表明：Config A（纯 LLM）可能产生环路，综合质量最低；Config B 加入 Rust 验证后，环路被拦截、FMEA 标注有了，但 d-分离问题只能检测不能修正；Config C（完整 EdgeThemis）在所有维度均表现最优。

5.3 案例分析

以医院感染场景为例。Config A 的输出存在 $D \rightarrow A$ 的环路，违反 DAG 约束，无 FMEA 标注。经过 Config C 的 3 轮自反思修正后，环路消除，每条边标注了 FMEA RPN 风险值，高风险边（如“败血症患者 $\rightarrow$ 死亡”，RPN=1010，S=9）被自动告警。

6 结论

本文介绍了 EdgeThemis——一个基于形式化验证的因果推理引擎。通过将 Rust 图论算子嵌入 LLM 推理循环，系统实现了以下突破：

1. 首次将 FMEA 工业风险评估与 LLM 因果推理结合，实现因果边级别的风险量化。
2. Bayesian Ball d-分离 + Kahn 环路检测实现轻量级形式化验证， $O(V + E)$ 环路检测 + $O(V^3)$ d-分离。
3. 自反思闭环显著提升因果图逻辑一致性，最多 5 轮自动修正。

已知局限包括： $O(V^3)$ 算法在节点数 $N > 50$ 时计算量呈立方级增长；边缘量化模型在多轮重试中可能出现 JSON 语法截断和 FMEA 评分“方差坍塌”。未来计划扩展至更大规模因果图、引入人类反馈机制、并支持时序因果推理。

参考文献

- [1] J. Chen et al., “Can Large Language Models Infer Causation from Correlation?,” in *Proc. ICLR*, 2024.
- [2] N. Shinn et al., “Reflexion: Language Agents with Verbal Reinforcement Learning,” in *Proc. NeurIPS*, 2023.
- [3] CausalFlip: A Benchmark for LLM Causal Judgment Beyond Semantic Matching, *arXiv:2602.20094*, 2025.
- [4] CRAwDAD: Causal Reasoning Augmentation with Dual-Agent Debate, *arXiv:2511.22854*, 2024.
- [5] Causal Agent based on Large Language Model, *arXiv:2408.06849*, 2024.

A 附录 A: Rust 核心代码

A.1 dag.rs —紧凑邻接表

Listing 1: CompactCausalGraph 结构体定义

```

1  #[derive(Debug, Clone)]
2  pub struct CompactCausalGraph {
3      pub adjacency_list: Vec<Vec<usize>>,
4      pub rev_adjacency_list: Vec<Vec<usize>>,
5      pub node_count: usize,
6  }
7
8  impl CompactCausalGraph {
9      pub fn new(initial_capacity: usize) -> Self {
10         CompactCausalGraph {
11             adjacency_list: vec![Vec::new(); initial_capacity],
12             rev_adjacency_list: vec![Vec::new(); initial_capacity],
13             node_count: 0,
14         }
15     }
16
17     pub fn add_edge(&mut self, from_id: usize, to_id: usize) {
18         self.ensure_capacity(from_id.max(to_id));
19         if !self.adjacency_list[from_id].contains(&to_id) {
20             self.adjacency_list[from_id].push(to_id);
21             self.rev_adjacency_list[to_id].push(from_id);
22         }
23     }
24 }

```

A.2 algorithms.rs —Kahn 环路检测（关键片段）

Listing 2: Kahn 拓扑排序环路检测

```

1  pub fn kahn_cycle_detect(graph: &CompactCausalGraph) -> bool {
2      let n = graph.node_count;
3      if n == 0 { return true; }
4      let mut in_degree = vec![0; n];

```

```

5     for from_node in 0..n {
6         if from_node < graph.adjacency_list.len() {
7             for &to_node in &graph.adjacency_list[from_node] {
8                 in_degree[to_node] += 1;
9             }
10        }
11    }
12    let mut queue: Vec<usize> = (0..n)
13        .filter(|&i| in_degree[i] == 0).collect();
14    let mut processed = 0;
15    while let Some(node) = queue.pop() {
16        processed += 1;
17        if node < graph.adjacency_list.len() {
18            for &neighbor in &graph.adjacency_list[node] {
19                in_degree[neighbor] -= 1;
20                if in_degree[neighbor] == 0 {
21                    queue.push(neighbor);
22                }
23            }
24        }
25    }
26    processed == n // true = 无环, false = 有环
27 }

```

A.3 algorithms.rs —Bayesian Ball d-分离（关键片段）

Listing 3: Bayesian Ball 方向状态 BFS 核心循环

```

1 pub fn is_d_separated(
2     graph: &CompactCausalGraph,
3     x: usize, y: usize,
4     observed: &HashSet<usize>
5 ) -> bool {
6     let rev_adj = graph.rev_adj();
7     // 预计算观测节点的祖先（用于 Collider 激活判断）
8     let mut ancestors_of_observed = HashSet::new();
9     let mut q = VecDeque::new();
10    for &obs in observed {
11        q.push_back(obs);
12        ancestors_of_observed.insert(obs);

```

```

13     }
14     while let Some(node) = q.pop_front() {
15         for &parent in &rev_adj[node] {
16             if ancestors_of_observed.insert(parent) {
17                 q.push_back(parent);
18             }
19         }
20     }
21     // 贝叶斯球开始滚动
22     let mut queue = VecDeque::new();
23     let mut visited = HashSet::new();
24     queue.push_back((x, false)); // false = Up
25     visited.insert((x, false));
26     while let Some((curr, is_down)) = queue.pop_front() {
27         if curr == y { return false; } // 球到终点 = 未分离
28         let is_start = curr == x;
29         // Rule 1/2: Up 方向
30         if !is_down && (is_start || !observed.contains(&curr)) {
31             for &child in &graph.adjacency_list[curr] {
32                 if visited.insert((child, true)) {
33                     queue.push_back((child, true));
34                 }
35             }
36         }
37         // Rule 4: Down + 已观测 → 允许 Up (Collider 激活)
38         else if is_down {
39             if is_start || ancestors_of_observed.contains(&curr)
40             {
41                 for &parent in &rev_adj[curr] {
42                     if visited.insert((parent, false)) {
43                         queue.push_back((parent, false));
44                     }
45                 }
46             }
47             // Rule 3: Down + 未观测 → 只允许 Down
48             if !is_down && (is_start || !observed.contains(&curr)) {
49                 for &parent in &rev_adj[curr] {
50                     if visited.insert((parent, false)) {
51                         queue.push_back((parent, false));

```

```

52         }
53     }
54     } else if is_down && (is_start || !observed.contains(&
        curr)) {
55         for &child in &graph.adjacency_list[curr] {
56             if visited.insert((child, true)) {
57                 queue.push_back((child, true));
58             }
59         }
60     }
61 }
62 true // 球没到 Y = 被 d-分离
63 }

```

A.4 fmea_evaluator.rs —FMEA RPN 评分

Listing 4: FMEA 风险评分

```

1  #[pyclass]
2  pub struct FmeaScore {
3      pub s: u32, pub o: u32, pub d: u32,
4  }
5
6  #[pymethods]
7  impl FmeaScore {
8      #[new]
9      pub fn new(s: u32, o: u32, d: u32) -> PyResult<Self> {
10         if s < 1 || s > 10 || o < 1 || o > 10 || d < 1 || d > 10
11         {
12             return Err(pyo3::exceptions::PyValueError::new_err(
13                 "S/O/D must be in range 1-10"
14             ));
15         }
16         Ok(FmeaScore { s, o, d })
17     }
18
19     pub fn calculate_rpn(&self) -> u32 {
20         (100 * self.s) + (10 * self.o) + self.d
21     }
22 }

```


B 附录 B: Python 管线代码

B.1 agent_state_machine.py —数据模型

Listing 5: Pydantic 数据模型定义

```

1 from pydantic import BaseModel
2 from typing import TypedDict, List
3
4 class CausalEdge(BaseModel):
5     cause: str
6     effect: str
7     S: int # Severity (1-10)
8     O: int # Occurrence (1-10)
9     D: int # Detection (1-10)
10
11 class ExtractedGraph(BaseModel):
12     nodes: List[str]
13     edges: List[CausalEdge]
14     reasoning_process: str
15
16 class ReflectorVerdict(BaseModel):
17     verdict: str # "PASS" or "REJECT"
18     reason: str
19
20 class CausalAgentState(TypedDict):
21     input_text: str
22     extracted_graph: dict
23     is_safe: bool
24     d_separation_claims: List[str]
25     rust_interception_report: str
26     interception_count: int

```

B.2 nodes.py —Validator 节点（关键片段）

Listing 6: Validator 节点: FMEA 风险告警逻辑

```

1 def validate_graph_node(state):
2     edges = state["extracted_graph"]["edges"]
3     # FMEA RPN 告警

```

```

4     fmea_alerts = []
5     for edge in edges:
6         rpn = 100 * edge.S + 10 * edge.O + edge.D
7         if rpn > 500:
8             fmea_alerts.append(
9                 f"高危_RPN:_{edge.cause}->{edge.effect}_{rpn}")
10            )
11        if edge.S >= 9:
12            fmea_alerts.append(
13                f"致命单点:_{edge.cause}->{edge.effect}_{S={edge.S}}")
14            )
15        # Kahn 环路检测
16        topology_safe = rust_engine.kahn_cycle_detect()
17        # Bayesian Ball d-分离
18        if topology_safe:
19            claims = rust_engine.extract_testable_claims()
20        return {"is_safe": topology_safe and not claims,
21               "d_separation_claims": claims}

```

B.3 app.py —LangGraph 状态机组装（关键片段）

Listing 7: LangGraph 状态机路由逻辑

```

1  from langgraph.graph import StateGraph, START, END
2
3  def route_after_validate(state):
4      if state.get("interception_count", 0) >= 5:
5          return END # 熔断
6      if not state.get("is_safe"):
7          if state.get("d_separation_claims"):
8              return "reflector"
9          return "generate" # 环路 → 直接重试
10     return END
11
12  def route_after_reflector(state):
13      if state.get("interception_count", 0) >= 5:
14          return END # 熔断
15      if state.get("verdict") == "REJECT":
16          return "generate"

```

```
17     return END
18
19 graph = StateGraph(CausalAgentState)
20 graph.add_node("generate", generate_graph_node)
21 graph.add_node("validate", validate_graph_node)
22 graph.add_node("reflector", reflector_node)
23 graph.add_edge(START, "generate")
24 graph.add_edge("generate", "validate")
25 graph.add_conditional_edges("validate", route_after_validate)
26 graph.add_conditional_edges("reflector", route_after_reflector)
```